

How to handle C2C-messages

How to use websocket to handle messages and reply:

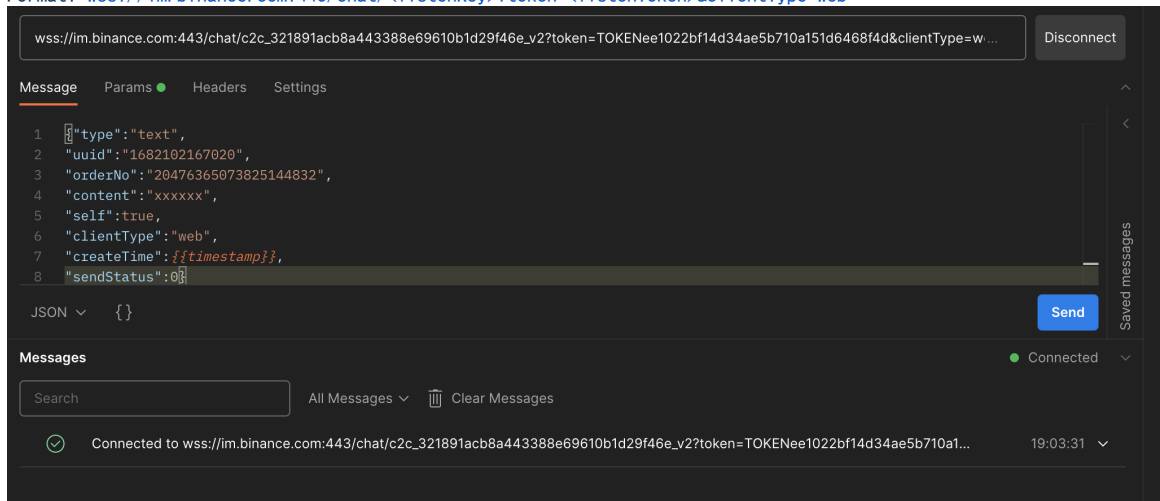
You can use `GET /sapi/v1/c2c/chat/retrieveChatCredential` to get credentials.

Response:

```
{
  "code": "000000",
  "message": "success",
  "data": {
    "chatWssUrl": "wss://im.binance.com:443/chat",
    "listenKey": "c2c_321891acb8a443388e69610b1d29f46e_v2",
    "listenToken": "TOKEN4e75573ca1e44e96ba809840d05d3e03"
  },
  "success": true
}
```

Using those endpoints you can create a websocket connection (on postman for example)

Format: `wss://im.binance.com:443/chat/<listenKey>?token=<listenToken>&clientType=web`



To send a new message (or reply income messages) you need follow this format:

```
{
  "type": "text",
  "uuid": "1682102167020", // random string
  "orderNo": "20476365073825144832", // use your own order number
  "content": "xxxxxx",
  "self": true,
  "clientType": "web",
  "createTime": {{timestamp}},
  "sendStatus": 0
}
```

Python code implementation:

```
import websocket
import json
import hmac
import time
import hashlib
import requests
from urllib.parse import urlencode
import json

BASE_URL = "https://api.binance.com" # production base url
```

```

KEY = "--"
SECRET = "----"

def hashing(query_string):
    return hmac.new(
        SECRET.encode("utf-8"), query_string.encode("utf-8"), hashlib.sha256
    ).hexdigest()

def get_timestamp():
    return int(time.time() * 1000)

def dispatch_request(http_method):
    session = requests.Session()
    session.headers.update(
        {"Content-Type": "application/json;charset=utf-8", "X-MBX-APIKEY": KEY, "clientType": "WEB"}
    )
    return {
        "GET": session.get,
        "DELETE": session.delete,
        "PUT": session.put,
        "POST": session.post,
    }.get(http_method, "GET")

# used for sending request requires the signature
def send_signed_request(http_method, url_path, payload={}, dataLoad={}):
    query_string = urlencode(payload)
    # replace single quote to double quote
    query_string = query_string.replace("%27", "%22")
    if query_string:
        query_string = "{}&timestamp={}".format(query_string, get_timestamp())
    else:
        query_string = "timestamp={}".format(get_timestamp())

    url = (
        BASE_URL + url_path + "?" + query_string + "&signature=" + hashing(query_string)
    )

    # print("{} {}".format(http_method, url))
    params = {"url": url, "params": {}, "data": dataLoad}

    response = dispatch_request(http_method)(**params)
    return response

uri_path = "/sapi/v1/c2c/chat/retrieveChatCredential"

dataLoad = {}
param = {}
response = send_signed_request("GET", uri_path, param, dataLoad)

print(response)
response = response.json()

wss_url = response['data']['chatWssUrl'] + "/" + response['data']['listenKey'] + "?token=" + \
    response['data']['listenToken'] + "&clientType=web"

# websocket.enableTrace(True)

def on_message(ws, message):
    print(json.loads(message))
    return

```


GET /sapi/v1/c2c/chat/retrieveChatMessagesWithPagination Retrieve Chat Messages with Pagination									
Parameters									
Name	Description	No.	Time	Request Content	Response Code	Call count	Test Plan	Operation	
chatMessageType string (query)	chat message type: text/image etc	1	04-26 17:22:52	preview	200	1	☒	delete	
	chatMessageType - chat message type: text/im	2	04-26 17:16:35	preview	200	1	☒	delete	
		3	04-26 17:14:26	preview	200	1	☒	delete	
clientType ★ required (header)	Client Device Type	4	04-26 17:10:51	preview	200	1	☒	delete	
	clientType - Client Device Type	5	04-26 17:10:20	preview	200	1	☒	delete	
id integer(\$int64) (query)	Query message ID, optional. It will be used as one of the query conditions, the queried message ID < the provided ID	6	04-26 17:08:42	preview	200	1	☒	delete	
	Id - Query message ID, optional. It will be used	7	04-26 17:07:35	preview	200	1	☒	delete	

Then use the following sample code to download picture (Python example)

```
import requests

def download_file(url, save_path):
    response = requests.get(url)
    if response.status_code == 200:
        with open(save_path, 'wb') as file:
            file.write(response.content)
            print("File downloaded successfully.")
    else:
        print(f"Failed to download file. Status code: {response.status_code}")

if __name__ == "__main__":
    file_url = "https://static.qa1fdg.net/client_upload/c2c/chat/20240426/e49f32b6b337445cbd5a43f427174194_20240426120643.jpg"
    save_file_path = "1_downloaded.jpg"
    download_file(file_url, save_file_path)
```